

myAssistente5

Desktop Personal Assistant – Documentation Toolkit

(1) Installation Manual · Release 5 · English



Home



Author

GB English

IT Italian

Contents: [Installation](#) [Procedure](#) [First steps](#) [Screenshots](#)
[Documentation](#)

(1) Installation Manual

- **Purpose:** This manual guides you through the first installation of your myAssistente.
- **Prerequisites:** The ZIP package containing the myAssistente executable for your operating system,
or alternatively: the Python files myAssistente.py and (optional, for AIML support) aiml_parser.py with Python installed on your machine (cross-platform).
- **Sections:** Installation, First program launch, Try it

Download all required files from the [Assistente project GitLab repository](#).

Italian and English languages are included in the Windows and Linux packages. If you need additional languages (15 ready-to-use languages), download the lang_XX.json file from the [localization-file folder](#) in the same repository.

Some example AIML files are also included; additional Italian AIML files can be found in the aiml-file folder of the same repository, or you may use any standard AIML files available online.

Installation Procedure

The myAssistente application is portable and can be launched from any location, including the desktop or a USB drive.

Windows

Download and extract `myAssistente[last release]windows.zip`

Run `myAssistente.exe`

Linux

Download and extract `myAssistente[last release]linux.zip`

Run `myAssistente`

Python

Download `myAssistente_[last release].py`

Download the optional AIML engine `aiml_parser.py`

Download the full `_dati/` folder

Ensure your Python installation includes all required dependencies (see README)

Place `myAssistente_[last release].py` , `aiml_parser.py` , and the `_dati/` folder in the same directory

Run `myAssistente_[last release].py`

All Operating Systems

Optional: download and place any additional language files `lang_XX.json` inside the `_dati` folder (the same location where `lang_it.json` and

lang_en.json are stored), and any AIML files inside
_dati/aiml/[language]/.

To customize the default language and other parameters, refer to the (2) *User Manual*. Many AIML tutorials are available online; the few optional parameters supported by myAssistente are described in the (6) *AIML Manual*.

File structure:

```
myAssistente  <- .py, .exe or no extension for Linux
aiml_parser.py <- required only for Python users
_dati/
    config.json
    memory.json
    memory.json.bak
    langIT.json <- default language
    langEN.json <- default language
    langXX.json <- additional languages
    asset/      <- your personal files will be saved here
        avatar/ <- avatar images and videos
        manuals/ <- HTML myAssistente manuals
    aiml/       <- optional AIML files
        IT/     <- AIML files for Italian
            saluti.aiml
        EN/     <- AIML files for English
            greetings.aiml
        XX/     <- create a folder for AIML in your language
```

First steps

In the (2) *User Manual* you will find step-by-step configuration instructions.

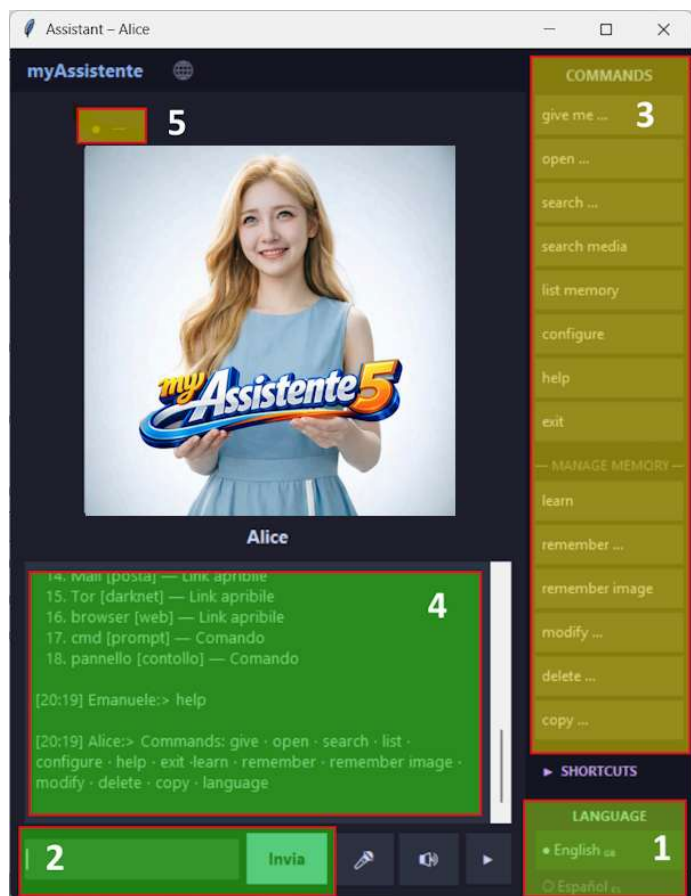
- 1) **Language** — after following the previous instructions, the language should already be correct.
- 2) **User name and assistant name** — how to display your user name and assign a name to your assistant.
- 3) **Training the assistant — DATA:** instructions to store a piece of data and then ask the assistant to open it.
- 4) **Training the assistant — LINKS:** instructions to store a link and then ask the assistant to open it.
- 5) **Shortcut buttons** — instructions to customize the quick-access buttons at the bottom right.

In the (6) *AIML Manual* you will find step-by-step instructions for the AIML language.

- 6) **Improving question understanding:**
After training step 3, to retrieve the stored data you must write `:> give me [data name]`.
After training step 4, to open the stored link you must write `:> open [link name]`.
But we can do better: for this purpose, myAssistente is compatible with the AIML language.

Screenshots

Welcome screen



1) Area to **change language**.

If the language is not correct at startup, change it using the buttons at the bottom right.

NOTE: the languages present in the `_dati` folder are detected automatically. The program starts with the last selected language.

2) Area to **enter commands**. By pressing the microphone you can use voice commands.

Note: STT (Speech to Text) uses **Google APIs**, and audio will be sent to Google. For maximum privacy, do not press the microphone: use text commands only.

3) **Command buttons** area. If you already know the commands, close this sidebar using the arrow at the bottom right, next to the speaker icon.

4) **Assistant response area**. Responses will be shown here and read aloud when audio is enabled. To stop the reading, enter the command `ok` or press the button in the SHORTCUT panel ("Quiet please").

5) Privacy control area

At the top left you will find the response source indicator, so you are always in control of your privacy.

In addition to when the  red warning dot appears, your voice commands are always sent externally (Google Speech API) without warning, in order to be converted into text. For 100% privacy **do not use the microphone function and do not enable AI APIs.**

 green — memory: response obtained internally from `memory.json`

 blue — AIML: response obtained internally from `aiml/langXX/[file].aiml`

 red — AI: response obtained from an external AI provider

 grey — system messages

Response search priority:

1)  grey: TTS stop words (`ok`, `stop`, `silence...`)

2)  grey: Active dialogue states (`remember`, `learn`, `delete`, `modify`, `configure...`)

3)  green: Fixed responses — from `config.json`

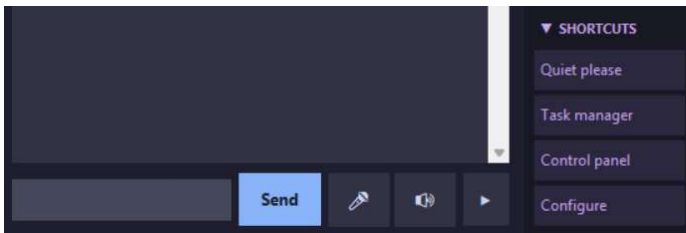
4)  green: Commands — from `memory.json`

5)  blue: AIML — from `.aiml` files

6)  red: External AI — OpenAI or Gemini (if enabled)

7)  grey: `_not_understood()` — final fallback system message

Shortcut



By clicking the arrow at the bottom left of SHORTCUT you can open/close the customizable quick-command panel.

All buttons in this area are customizable. For details, see the *(2) User Manual*.

Default commands

Command	Description (lang_en.json)
<code>find... <name></code>	Find data in memory
<code>open... <name></code>	Open files, folders or programs
<code>search... <query></code>	Search in memory
<code>search media <query></code>	Search images, music and videos in the _dati/asset folder
<code>list memory</code>	Show all memory entries
<code>configure</code>	Configure the config.json file
<code>help</code>	List available commands
<code>exit</code>	Close the myAssistente program
--- MEMORY MANAGEMENT ---	
<code>-</code>	
<code>learn</code>	Guided input
<code>remember... <name></code> <code><data></code>	Save information in memory
<code>remember image <name></code> <code><data></code>	Save images in memory
<code>modify... <name></code>	Edit an entry

Command	Description (lang_en.json)
delete... <name>	Delete an entry
copy... <name>	Copy an entry to the clipboard
SHORTCUT	
<i>list quick commands</i>	
LANGUAGE	
language name or code	Change language
<code>	

Note All command names and button labels in the assistant window can be customized in the `config.json` file. For each command you may define multiple alternative words. For more information read the *(3) Configuration Manual*.

Documentation

The documentation available for **myAssistente5**.

The documents can also be found in the [/DOCS](#) folder of the repository. For any missing documentation, refer to the English or Italian documentation of previous releases.

- **(1) Installation Manual** — installation of the program for Windows, Linux or cross-platform
- **(2) User Manual** — myAssistente user interface
- **(3) Configuration Manual** — `config.json` advanced settings
- **(4) Memory Manual** — `memory.json` advanced settings
- **(5) Language Manual** — `lang[].json` advanced customization
- **(6) AIML Manual** — `aiml/[LANG]/[filename].aiml`
- **(7) Advanced AIML Manual** — `aiml/IT/testConiglio.aiml`

Toolkit **myAssistente5** – Desktop personal assistant

[Author's website](#) · © 2026 Emanuele Cassani